Broken by Default: A Formal Verification Study of Security Vulnerabilities in AI-Generated Code

Empirical Analysis of 1,000 Code Artifacts Across Five Large Language Models Using Z3 SMT Solver, Ablation Studies, and Static Tool Comparison

Dominik Blain, *Founder*
Maxime Noiseux, *Co-Founder*

Cobalt AI
Gatineau, QC, Canada
dominik@qreativelab.io

April 4, 2026

Abstract

We present *Broken by Default*, a systematic empirical study of security vulnerabilities in code generated by five major large language models (LLMs): GPT-4o, Claude Haiku 4.5, Gemini 2.5 Flash, Mistral Large, and Llama 3.3 70B. Using a benchmark of 200 security-critical prompts spanning five CWE categories—memory allocation (CWE-131/190), integer arithmetic (CWE-190/195), authentication (CWE-916), cryptography (CWE-327/330), and injection vulnerabilities (CWE-89/22/78), 40 prompts per category—we collected 1,000 code artifacts and subjected each to formal verification via the Z3 SMT solver within the COBALT analysis pipeline.

Our findings reveal a mean vulnerability rate of **48.8%** across all models. GPT-4o produced the highest vulnerability rate at 60.5% (grade F); Gemini 2.5 Flash performed best at 22.5% (grade B). A total of **294 findings** were formally proven exploitable via Z3 satisfiability witnesses, and 6 of 7 selected vulnerabilities were confirmed with real memory corruption crashes using GCC AddressSanitizer (ASAN). We further conduct three additional experiments on a v1 50-prompt sub-corpus: (1) a secure prompt ablation showing that explicit security instructions reduce the mean vulnerability rate by only 4 percentage points (to 60.8%), leaving four of five models at grade F; (2) a static tool comparison showing that six industry-standard tools together detect fewer than 7.6% of all artifacts and miss 97.8% of Z3-proven vulnerabilities, demonstrating a structural detection gap; and (3) a generation–review asymmetry experiment showing that models identify their own Z3-proven vulnerabilities 78.7% of the time in review mode (70 of 89 artifacts)—yet generate them at 48.8% by default. Our results suggest that current LLMs produce security-deficient code by default, that security instructions are insufficient mitigations, and that formal verification is the only approach capable of establishing ground-truth exploitability at scale.

1 Introduction

The rapid adoption of AI-assisted coding tools has fundamentally changed software development workflows. GitHub Copilot reported over 1.8 million paid subscribers by 2024 [1]. ChatGPT, Claude, and Gemini are routinely used to generate production code across security-sensitive domains including web backends, embedded systems, and cryptographic libraries.

Yet the security properties of AI-generated code remain understudied. Prior work [2–4] has identified categories of vulnerabilities in LLM outputs, but most rely on static pattern matching or manual inspection—techniques that cannot definitively establish exploitability.

Our contribution is methodological: we apply formal verification via the Z3 Satisfiability Modulo Theories (SMT) solver to establish ground-truth exploitability of AI-generated code vulnerabilities. Where a Z3 witness exists, the vulnerability is not a potential issue—it is a proven one, with a concrete input that triggers the fault condition. We further validate selected findings with compiler-instrumented runtime crashes using GCC AddressSanitizer.

Our findings (200-prompt v2 corpus) reveal a mean vulnerability rate of 48.8% across all models. GPT-4o leads at 60.5% (grade F); Gemini 2.5 Flash performs best at 22.5% (grade B). A total of 294 findings were

formally proven exploitable via Z3 satisfiability witnesses, and 6 of 7 selected vulnerabilities were confirmed with real memory corruption crashes using GCC AddressSanitizer (ASAN). We further conduct three additional experiments on the v1 50-prompt corpus: (1) a secure prompt ablation showing that explicit security instructions reduce the mean vulnerability rate by only 4 percentage points (to 60.8%), leaving four of five models at grade F; (2) a static tool comparison showing that six industry tools combined flag only 7.6% of artifacts, missing 97.8% of formally Z3-proven vulnerabilities; and (3) a generation–review asymmetry experiment showing that models identify their own Z3-proven vulnerabilities 78.7% of the time in review mode (70/89)—yet generate them at 48.8% by default. Our results suggest that current LLMs produce security-deficient code by default, that security instructions are insufficient mitigations, and that formal verification is the only approach capable of establishing ground-truth exploitability at scale.

1.1 Research Questions

- **RQ1:** At what rate do leading LLMs produce vulnerable code when prompted for security-sensitive tasks?
- **RQ2:** How do vulnerability rates and severity distributions differ across models?
- **RQ3:** Can Z3 SMT witnesses be operationalized into real runtime exploits?
- **RQ4:** Do explicit security instructions in the system prompt reduce vulnerability rates?
- **RQ5:** How does COBALT's detection rate compare to industry-standard static analysis tools?
- **RQ6:** Do models possess the knowledge to detect their own generated vulnerabilities?

1.2 Scope and Limitations

This study targets a bounded set of 200 prompt templates (40 per CWE category), each designed to elicit code in a security-sensitive domain. Results reflect model behavior on this prompt corpus as of April 2026 and should not be extrapolated to all possible LLM usage patterns. Models were queried via their production APIs at temperature 0 (deterministic mode). Prompt templates are released alongside this paper.

2 Related Work

Pearce et al. (2022) [2] evaluated GitHub Copilot on 89 scenarios across 18 CWEs and found 40% of suggestions contained vulnerabilities. Their analysis relied on CWE pattern matching without formal exploitability proofs.

Sandoval et al. (2023) [3] studied security of Copilot suggestions for C code, finding that users who accepted suggestions without review introduced more vulnerabilities than those who wrote code from scratch.

Tony et al. (2023) [4] introduced LLMSecEval, a benchmark of 150 natural-language prompts and evaluated 5 LLMs, finding high rates of insecure code generation particularly for injection and buffer-related vulnerabilities.

Perry et al. (2023) [5] conducted a user study where participants using AI assistants wrote significantly more security vulnerabilities than those who did not, despite expressing higher confidence in their code.

Meta CyberSecEval (2024) [11] benchmarks LLMs on generating exploit code and assisting cyberattacks. Their focus is adversarial capability—can the model help an attacker? Our focus is orthogonal: does the model introduce vulnerabilities in everyday code generation, without any adversarial intent?

SecurityEval (2022) [12] provides 130 Python prompts derived from CWE patterns and evaluates GitHub Copilot. Like Pearce et al., evaluation relies on pattern-based analysis. No formal exploitability proof is provided.

SVEN (2023) [13] trains a security vector to steer code generation toward secure outputs. Their setup assumes the vulnerability exists; ours measures baseline generation without steering intervention.

Our work differs in four key dimensions: (1) we apply formal Z3 SMT verification rather than pattern matching—producing mathematical proof of exploitability with concrete witness values; (2) we evaluate five current frontier models simultaneously across identical prompts; (3) we provide runtime exploitability confirmation via ASAN; and (4) we quantify the detection gap against six industry tools, including heavyweight C analyzers and CodeQL, demonstrating that 97.8% of formally proven vulnerabilities are structurally invisible to existing tooling.

3 Methodology

3.1 Benchmark Design

We constructed a benchmark of 200 natural-language prompts distributed across five CWE categories (40 prompts each):

- **MEM** (CWE-131/190): Buffer allocation, array indexing, dynamic memory management in

C/C++

- **INT** (CWE-190/195): Integer arithmetic, type casting, signed/unsigned arithmetic
- **AUTH** (CWE-916): Password hashing, authentication token generation, session management
- **CRYPTO** (CWE-327/330): Key generation, random number usage, cipher selection
- **INP** (CWE-89/22/78): SQL construction, file path handling, shell command composition

Prompts were designed to be representative of real developer tasks—not adversarial jailbreaks. Example prompt (MEM-01):

> *"Write a C function that reads n integers from a file descriptor and returns them as a dynamically allocated array."*

3.2 Model Selection and API Configuration

Five models were evaluated, selected to represent the frontier of publicly accessible LLMs as of Q1 2026:

Model	API Identifier
GPT-4o	`gpt-4o`
Claude Haiku 4.5	`claude-haiku-4-5-20251001`
Gemini 2.5 Flash	`gemini-2.5-flash`
Mistral Large	`mistral-large-latest`
Llama 3.3 70B	`llama-3.3-70b-versatile`

All models were queried at temperature 0 to ensure reproducibility. System prompts instructed models to produce complete, runnable code without additional commentary, maximizing the proportion of testable output.

3.3 COBALT Analysis Pipeline

Generated code artifacts were analyzed using the COBALT static analysis engine, which combines:

1. **CWE Pattern Extraction**: Regex and AST-level patterns identify candidate vulnerability sites (e.g., `malloc(n * sizeof(T))` without overflow guard)
2. **Z3 SMT Encoding**: Vulnerability conditions are encoded as Z3 formulas. For integer overflow: given n as a free variable of type `BitVec(32)`, check satisfiability of $n \times sizeof(T) < n$ (overflow condition under unsigned 32-bit semantics)
3. **Witness Extraction**: When Z3 returns SAT, the concrete witness value (e.g., $n = 2^{30} + 1$) is extracted and classified as the exploit input

Findings were classified as:

- **Z3 SAT**: Formally proven exploitable via SMT witness
- **PATTERN MATCH**: Structural vulnerability identified, Z3 encoding pending
- **CLEAN**: No vulnerability detected

Severity was assigned using CVSS v3 base score criteria: CRITICAL (≥9.0), HIGH (7.0–8.9), MEDIUM (4.0–6.9).

3.4 Runtime Exploitability Validation

To counter the objection that pattern-based detection inflates vulnerability counts, we selected 7 representative findings and constructed C and Python proof-of-concept (PoC) harnesses. C PoCs were compiled with:

```
gcc -fsanitize=address,undefined -g -O1 \
  -o poc poc.c && ./poc
```

Z3-extracted witness values were used as concrete inputs. We captured ASAN output to confirm runtime faults.

4 Results

4.1 RQ1: Overall Vulnerability Rates

Table 1 presents aggregate results across all 200 prompts per model.

Table 1: Benchmark Leaderboard — 200 Prompts per Model

Model	Vuln	CRIT	HIGH	Z3	Grd
GPT-4o	60.5%	69	41	65	F
Llama 3.3 70B	59.0%	74	33	61	D
Claude Haiku 4.5	54.5%	69	35	61	D
Mistral Large	47.5%	53	32	63	D
Gemini 2.5 Flash	22.5%	37	8	44	B
Mean	48.8%	60.4	29.8	58.8	—

Grades: A <10%, B 10–29%, C 30–44%, D 45–59%, F ≥60% vulnerability rate.

The mean vulnerability rate of 48.8% is alarmingly high. GPT-4o led with 121 vulnerable outputs out of 200 (60.5%). CRITICAL-severity findings dominated: an average of 60 CRITICAL findings per model. Integer arithmetic (INT) produced the highest category rate at 80%, followed by memory allocation (MEM) at 69%.

4.2 RQ2: Vulnerability Distribution by CWE

Table 2 illustrates that memory safety vulnerabilities (CWE-131, CWE-190) accounted for the largest share of findings across all models, consistent with known challenges in generating safe C/C++ code.

Table 2: Vulnerability Rate by Category (Mean Across Models)

Category	CWE(s)	Mean Rate
Memory Allocation	131, 190	69%
Integer Arithmetic	190, 195	80%
Authentication	916	33%
Cryptography	327, 330	25%
Input Handling	89, 22, 78	38%

Integer arithmetic prompts produced the highest vulnerability rate (80%), followed by memory allocation (69%), both driven by consistent failure to guard against integer overflow in malloc size computations and signed/unsigned conversion errors. A representative pattern found across all five models:

Listing 1: Typical vulnerable pattern (all 5 models)

```
// CWE-190: No overflow guard
int* buf = malloc(n * sizeof(int));
```

The safe pattern requires an explicit overflow check:

Listing 2: Correct pattern

```
if (n > SIZE_MAX / sizeof(int)) return
    NULL;
int* buf = malloc(n * sizeof(int));
```

None of the five models consistently generated the safe pattern across all MEM prompts.

4.3 RQ3: Runtime Exploitability Confirmation

Of 7 selected vulnerabilities subjected to PoC harness testing, 6 of 7 produced confirmed runtime faults. Table 3 summarizes the results.

Representative ASAN output for MEM-01-A (Llama, CWE-131):

Listing 3: ASAN output – MEM-01-A heap overflow

```
==AddressSanitizer: heap-buffer-overflow
WRITE of size 4 at 0x...
#0 poc_main (poc+0x...)
#1 main (poc+0x...)
shadow bytes around the buggy address:
0x...: fa fa fa fa fa fa fa fa
0x...: 00 00 00 00 00[fa]fa fa
SUMMARY: AddressSanitizer: heap-buffer-
    overflow
```

Table 3: PoC Exploit Confirmation Results

ID	Model	Fault Type	Result
MEM-01-A	Llama	heap-buf-overflow	✓
MEM-01-B	GPT-4o	heap-buf-overflow	✓
MEM-03	Llama	alloc-size-too-big	✓
MEM-06	GPT-4o	OOB read	✓
AUTH-03	Llama	SHA-256 crack	✓
INP-01	Mistral	SQL injection	✓
INP-06	GPT-4o	Zip Slip (path trav.)	blocked†

† Python 3.12 raises `ValueError` on path traversal. Vulnerable pattern present; blocked at runtime, not at generation.

For AUTH-03 (SHA-256 password cracking), a 6-character lowercase password was recovered from its hash in 0.01ms using a precomputed lookup, demonstrating that CWE-916 (insufficient password hashing) is not merely theoretical.

SQL injection in INP-01 produced a working query that extracted all records including a synthetic credit card number embedded in the test database, confirming complete data exfiltration.

4.4 RQ4: Secure Prompt Ablation

We re-ran all 50 prompts of the v1 corpus for all five models with a security-explicit system prompt instructing models to apply security best practices, guard against integer overflow, and produce production-ready code. Note: the v1 corpus was collected with Claude 3.5 Sonnet; the v2 leaderboard uses Claude Haiku 4.5, so per-model rates are not directly comparable across tables. Table 4 shows the results. (Ablation on the full 200-prompt corpus is left for future work.)

Table 4: Baseline vs. Secure Prompt Ablation (v1 corpus, 50 prompts). †Claude row reflects v1 data collected with Claude 3.5 Sonnet; the v2 leaderboard uses Claude Haiku 4.5.

Model	Base	Secure	Δ	Grade
Llama 3.3 70B	68%	70%	+2%	F
GPT-4o	66%	58%	−8%	F
Gemini 2.5 Flash	60%	60%	0%	F
Claude 3.5 Sonnet†	64%	62%	−2%	F
Mistral Large	66%	54%	−12%	D
Mean	64.8%	60.8%	−4%	—

Explicit security instructions reduced the mean vulnerability rate by only 4 percentage points—from 64.8% to 60.8%. Four of five models remained at grade F. Llama 3.3 70B performed worse under the secure prompt (+2%). The improvement was category-

dependent: authentication and cryptography showed modest gains, while memory allocation vulnerabilities were essentially unchanged across all models—suggesting that security instructions do not override low-level memory management patterns learned from training data.

4.5 RQ5: Static Tool Comparison

We conducted three tool comparison experiments on the v1 50-prompt corpus (250 artifacts). First, we ran Semgrep (all rulesets) and Bandit across all 250 artifacts. Second, we ran three heavyweight C static analyzers—Cppcheck 2.13, Clang Static Analyzer, and FlawFinder 2.0—against all 87 C artifacts. Third, we ran CodeQL (security-extended query suite, version 2.x) against the 90 Z3 SAT artifacts to assess detection of formally proven findings. Table 5 summarizes all three experiments.

Of the 162 COBALT findings, 90 are Z3 SAT—formally proven with concrete arithmetic witnesses—and 72 are PATTERN MATCH, flagged by COBALT's AST layer without Z3 confirmation.

Pattern tools (Experiment A). All 19 tool-caught artifacts overlap with COBALT's PATTERN MATCH tier or dangerous-function patterns (`strcat`, `rand`), never with the integer overflow class. Of the 90 Z3 SAT findings, tools caught only 2—both via Semgrep's `insecure-use-strcat-fn` rule applied to MEM-02 (CWE-190) code that also contained `strcat`, not because any tool detected the integer overflow in the length computation. 97.8% (88/90) of formally Z3-proven vulnerabilities are invisible to all industry tools combined.

Heavyweight C analyzers (Experiment B). Cppcheck and Clang Static Analyzer detected zero of 80 confirmed C vulnerabilities. FlawFinder flagged 4 artifacts: 2 for `strcat` (MEM-02, CWE-190) and 2 for weak randomness (`rand`, CRYPTO-08, CWE-338)—no detection of integer overflow in allocation arithmetic. Combined, heavyweight analyzers missed 96.9% of COBALT findings on C.

This is not a configuration issue. We ran Cppcheck with `--enable=all --inconclusive --check-level=exhaustive`; only 113 of 592 available checkers activate on standard C11 code. Detecting `malloc(n * sizeof(T))` as an integer overflow requires arithmetic semantic reasoning under 32-bit modular arithmetic—which Z3 provides and which neither pattern matching nor path-sensitive analysis can perform without concrete value constraints or explicit taint propagation from attacker-controlled inputs.

CodeQL v2.25.1 (Experiment C). CodeQL (security-extended query suite, v2.25.1, native x86-64 on GitHub Actions) was run against all 90 formally Z3-proven findings: 68 C artifacts and 22 Python artifacts. CodeQL detected **0 of 68 C artifacts** and **0 of 22 Python artifacts**—a 100% miss rate across both languages (0/90 total). This result is consistent with Experiments A and B: integer overflow in allocation arithmetic (`malloc(n * sizeof(T))`) is structurally undetectable by dataflow and taint-tracking analysis without concrete arithmetic reasoning. CodeQL's integer overflow queries require explicit source-to-sink taint paths or known unsafe patterns—a property that Z3's bit-vector arithmetic encodes directly but that CodeQL's query model does not generalize to.

4.6 RQ6: The Generation–Review Asymmetry

A plausible counter-argument to our findings is that models generate vulnerable code simply because they are not asked to think about security. To test this, we conducted a self-review experiment: we fed each model's Z3-proven vulnerable code back to the same model and asked it to review the code for security vulnerabilities.

We fed all 89 valid Z3-proven artifacts back to their generating model and asked it to review the code for security vulnerabilities. (One artifact was excluded due to a Mistral API timeout during the review pass.) Of 89 artifacts reviewed, **70 of 89 (78.7%)** were correctly identified as vulnerable by the generating model. Table 6 presents per-model results.

This finding is more damning than a simple false-negative result. It reveals an *observed generation–review asymmetry*: models possess the knowledge to identify these vulnerability classes when in review mode, yet consistently fail to apply that knowledge during code generation. The problem is not a lack of security knowledge—it is a failure of spontaneous application at a 21.3% false-negative rate across the full artifact set. This suggests that RLHF or instruction fine-tuning aimed at security-conscious code review does not transfer reliably to the code generation task.

5 Discussion

5.1 Why Do Models Fail Consistently?

The consistency of failures—particularly in memory allocation—suggests a training data effect. C code on the internet overwhelmingly lacks overflow guards for malloc computations. Models appear to have internalized this pattern as correct. The secure prompt

Table 5: Detection Rate: COBALT vs. Six Industry Tools

Tool	Det. Rate	C / Py	Note
COBALT Z3	162/250 (64.8%)	80/87 · 82/163	
Z3 formally proven	90/250 (36.0%)		Pattern-based tools (Exp. A):
Semgrep (all rulesets)	16/250 (6.4%)	2/87 · 14/163	
Bandit (medium+ severity)	5/250 (2.0%)	— · 5/163	
Combined A	19/250 (7.6%)	—	
Heavyweight C analyzers (Exp. B, C only):			
Cppcheck 2.13	0/87 (0.0%)	0/87	
Clang Static Analyzer	0/87 (0.0%)	0/87	
FlawFinder 2.0 (risk≥3)	4/87 (4.6%)	4/87	strcat + rand only
Combined B	4/87 (4.6%)	—	
Z3 SAT formal proof set (Exp. C):			
CodeQL v2.25.1 (security-extended)	0/90 (0.0%)	0/68 · 0/22	100% miss; C + Python
COBALT-only (vs. A)	151/162 (93.2%)	78/80 · 73/82	
COBALT-only (vs. B)	157/162 (96.9%)	76/80 · 81/82	
Z3 SAT-only (vs. A+B+C)	88/90 (97.8%)		formal proofs

Table 6: Self-Review Experiment — Per-Model Results (N=89)

Model	Detected	Rate	FN Rate
Mistral Large	17/17	100%	0%
Llama 3.3 70B	14/17	82%	18%
Gemini 2.5 Flash	14/18	78%	22%
Claude 3.5 Sonnet†	13/19	68%	32%
GPT-4o	12/18	67%	33%
Total	70/89	78.7%	21.3%

† Self-review conducted with the same model used for v1 generation: Claude 3.5 Sonnet (`claude-3-5-sonnet-20241022`). The v2 leaderboard uses Claude Haiku 4.5.

ablation (RQ4) reinforces this hypothesis: even explicit instructions to guard against overflow had no measurable effect on memory allocation prompts.

5.2 Security Instructions Are Insufficient

A mean improvement of 4 percentage points under an explicit security prompt is not a safety margin—it is noise. Developers who add "write secure code" to their prompt prefixes should not interpret this as a meaningful mitigation. Our results suggest that the vulnerability patterns are baked into the model's generation prior, not into its instruction-following surface.

5.3 The COBALT Detection Gap

COBALT's detection advantage on the v1 corpus—64.8% vs. 6.4% for all Semgrep rulesets and 0% for Cppcheck and Clang SA on C—is not an artifact of overly sensitive rules. Of the 90 Z3 SAT findings (formally proven with concrete witnesses), 97.8% are invisible to all six tools. The 2 Z3 SAT cases tools did catch (MEM-02, CWE-190) were flagged by Semgrep's `strcat` rule—detecting a dangerous string function in the same code, not the integer overflow that Z3 proved exploitable. CodeQL v2.25.1, despite its reputation as a heavyweight semantic analyzer, likewise detected zero of 90 Z3-proven findings (0/68 C, 0/22 Python). No tool, across any configuration, detected a single integer overflow in malloc size computations. The detection gap reflects a structural limitation: syntactic tools match patterns in code; path-sensitive tools track values along known execution paths. Neither approach can determine that `malloc(n * sizeof(T))` is dangerous unless n is provably bounded—a property that requires arithmetic reasoning across the full domain of integer inputs, which Z3's bit-vector arithmetic encodes directly.

5.4 Implications for Developers

1. Treat AI-generated C/C++ as unreviewed code requiring explicit security audit
2. Apply `-fsanitize=address,undefined` to all AI-generated systems code
3. Do not rely on security prompt prefixes as a

substitute for review

4. Do not rely on Semgrep, Bandit, Cppcheck, Clang SA, FlawFinder, or CodeQL alone to catch AI-generated integer overflow in allocation arithmetic
5. Use formal verification or human review for allocation arithmetic

5.5 Limitations

Our benchmark of 200 prompts covers five CWE categories; other vulnerability categories may show different rates. All models were tested at temperature 0; higher temperatures may yield different distributions. Of the v2 corpus, 294 findings are Z3 SAT (formally proven). The v1 self-review experiment (89 valid Z3-proven artifacts) showed per-model false-negative rates ranging from 0% to 33%. The secure prompt ablation showed a 2-point increase for one model (Llama) on the v1 50-prompt corpus, which may be within noise. We did not test multi-turn interactions or retrieval-augmented generation contexts. Ablation experiments on the full 200-prompt corpus are left for future work. The public dataset (https://github.com/dom-omg/bbd-dataset) contains 826 of 1,000 artifacts; 174 artifacts (Mistral: 36, Gemini: 138) were not persisted due to API errors during the collection run. All 1,000 responses were analyzed in-process and results are fully reflected in Table 1; only the raw code artifacts are missing from the public release.

5.6 Ethical Considerations

All artifacts were generated under controlled conditions. PoC exploits targeted only locally generated test programs. No production systems were accessed. All prompts, results, and scripts are released to support reproducibility and independent replication. The full dataset of 1,000 artifacts with Z3 labels is publicly available at https://github.com/dom-omg/bbd-dataset.

6 Conclusion

We presented a multi-experiment formal verification study of AI-generated code security, evaluating five frontier LLMs across 1,000 code artifacts (200 prompts $\times$ 5 models) and three experimental conditions. Our findings converge on a single conclusion: AI coding assistants are broken by default.

The v2 benchmark (200 prompts) reveals a mean vulnerability rate of 48.8% and 294 Z3-proven findings—GPT-4o leads at 60.5%, Gemini performs best at 22.5%. Integer arithmetic prompts drive the highest failure rate (80%), confirming that overflow in allocation arithmetic is a persistent, model-agnostic weakness. On the v1 50-prompt corpus, the baseline vulnerability rate of 64.8% drops to only 60.8% under explicit security instructions—a 4-point reduction that leaves four of five models at grade F. Six industry-standard tools combined flag only 7.6% of v1 artifacts and miss 97.8% of formally Z3-proven vulnerabilities—not because of misconfiguration, but because integer overflow in allocation arithmetic is structurally undetectable by pattern and path-sensitive analysis. Models that generate vulnerable code correctly identify those vulnerabilities in review mode 78.7% of the time—yet generate them 48.8% by default—confirming a persistent generation–review asymmetry that explicit security prompting does not resolve.

Formal verification via Z3 SMT solving is not a niche research tool. It is the only approach that can establish ground-truth exploitability of the vulnerability patterns that dominate AI-generated code. This methodology should be a mandatory component of any AI code review pipeline operating on memory-sensitive or security-critical targets.

Future work will extend ablation experiments to the full 200-prompt corpus, add multi-language targets (Go, Rust, JavaScript), and evaluate the effect of targeted security fine-tuning on these specific CWE classes.

Reproducibility

Benchmark prompts, raw results (JSON), analysis scripts, and PoC harnesses are available at: https://github.com/dom-omg/broken-by-default

References

[1] GitHub. (2024). GitHub Copilot: The AI Coding Assistant. https://github.com/features/copilot

[2] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," *IEEE Symposium on Security and Privacy (S&P)*, pp. 754–768, 2022.

[3] G. Sandoval, H. Pearce, T. Nys, R. Karri, B. Dolan-Gavitt, and S. Garg, "Lost at C: A user study on the security implications of large language model code assistants," *USENIX Security Symposium*, 2023.

[4] C. Tony, M. Mutas, N.E.D. Ferreyra, and R. Scandariato, "LLMSecEval: A dataset of natural language prompts for security evaluations," *IEEE/ACM Mining Software Repositories (MSR)*, 2023.

[5] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" *ACM CCS*, pp. 2785–2799, 2023.

[6] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," *TACAS*, LNCS 4963, pp. 337–340, 2008.

[7] MITRE. (2024). CWE-131: Incorrect Calculation of Buffer Size. https://cwe.mitre.org/data/definitions/131.html

[8] MITRE. (2024). CWE-190: Integer Overflow or Wraparound. https://cwe.mitre.org/data/definitions/190.html

[9] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," *USENIX ATC*, 2012.

[10] MITRE. (2024). CWE-916: Use of Password Hash With Insufficient Computational Effort. https://cwe.mitre.org/data/definitions/916.html

[11] M. Bhatt et al., "CyberSecEval: A Comprehensive Evaluation Framework for Measuring Cybersecurity Risks of Large Language Models," *arXiv:2312.04724*, 2024.

[12] M.L. Siddiq and J.C.S. Santos, "SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques," *ACM MSR4PS Workshop*, 2022.

[13] J. He and M. Vechev, "Large Language Models for Code: Security Hardening and Adversarial Testing," *ACM CCS*, pp. 1865–1879, 2023.